Selective Recovery for Cascading Failures in Multi-Agent LLM Systems

Lucas Yan · Atharv Nair · Utkarsh Ranjan · Ethan Chen · Bijin Kurian Jiby

CSE 291A Phase 1 Report | Option 5: Failure Propagation and Recovery

1. Introduction

A multi-agent workflow can carry one mistake through several stages: a researcher selects the wrong hotel, a planner builds an itinerary around it, and an executor makes the booking. Restarting may repair the task, but also repeats work that was already correct. We ask: **given a known fault location, how much work can recovery preserve without sacrificing correctness?**

In Phase 1, we built a fault-injection harness and evaluated travel and coding workflows using Qwen2.5-7B-Instruct, GPT-OSS 120B, and Muse Glimmer 30B. Our *coarse recovery* baseline uses recorded dependencies to rerun affected steps and preserve independent work. We are exploring a monitor as a recovery orchestrator that assigns repair subtasks. The current prototype is a basic *fault-note injector*: it informs rerun agents about the fault but does not yet orchestrate recovery.

2. Related Work

Related work spans failure diagnosis, error propagation, and recovery. Together, these studies motivate our focus on preserving valid work when repairing a failed multi-agent workflow.

On diagnosis and propagation, Cemri et al. organize multi-agent failures into a taxonomy covering system design, coordination, and agent behavior [1]. Zhang et al. study which agent and step are responsible for a failed run [2]. Xie et al. model collaboration through dependencies to explain how one error spreads across agents [3]. These studies motivate our provenance record, but diagnosing a failure leaves a separate question: which completed results should recovery discard?

A complementary line of work examines recovery and state repair. UndoBench distinguishes successful task execution from recovery after failure [4]; our evaluation additionally measures how much work replay preserves. Sagas uses compensating actions to address committed effects [5], informing our distinction between restoring files, cancelling bookings, and recording irreversible emails. **Closest to our approach, Wang et al. repair faulty memories by tracing dependent actions and preserving results with independent support [6]. Our support variant implements a limited evidence-link rule, not their full sufficiency check.**

3. Methodology

3.1 Dependency recording and replay

The system records each step's inputs, outputs, and tool actions in a dependency graph. When a fault is reported, coarse recovery selects the step that produced it and later steps connected through the graph, preserving independent work. Before replay, it restores affected files and cancels bookings; sent emails cannot be undone. Selected steps run again without fault injection, while unaffected steps reuse saved outputs without model calls. An executor step is reused only if its planned action is unchanged; otherwise, it is recomputed and outdated effects are cleaned up.

We compare three replay policies:

- **Restart:** rerun the entire workflow.
- **Suffix:** rerun the faulty producer and every later step in execution order.
- **Coarse lineage:** rerun the faulty producer and reachable dependents; reuse eligible steps outside that set.

3.2 Monitor prototype

To explore fault-aware recovery, the current monitor prototype acts as a fault-note injector. It retains coarse lineage's restart set. It builds `FAULT_JSON` from the faulty artifact, producer, and stored label, instructing rerun agents to discard the bad output and redo the step (Figure 1). It adds prompt tokens but no model call.

We also evaluate a support variant that follows evidence-support links to preserve independently supported steps. Appendix C gives supplementary results.

4. Experimental Setup

4.1 Workloads and code evaluation

Travel booking has seven steps: flight/hotel research, planning, two bookings, itinerary writing, and email. Simulated tools let us check bookings, itinerary IDs, and recipients.

Coding uses specification, code, test, review, documentation, then a second specification–code–test–review pass. We evaluate ten of 50 fixed sanitized MBPP tasks [7], each requesting a Python function. One test provides revision feedback; hidden tests score only the final function. Code runs in isolated Docker containers (one CPU, 256 MB RAM, 15-second limit).

4.2 Models and execution settings

Qwen2.5-7B-Instruct (Q4_K_M) runs via llama.cpp on an M4 Pro MacBook Pro (20-core GPU, 24 GB RAM), one request at a time. GPT-OSS 120B and Muse Glimmer 30B use NRP and TritonAI APIs with five tasks in parallel. Hosted calls use temperature 0, seed 0, a 2,048-token output limit, and a 120-second timeout.

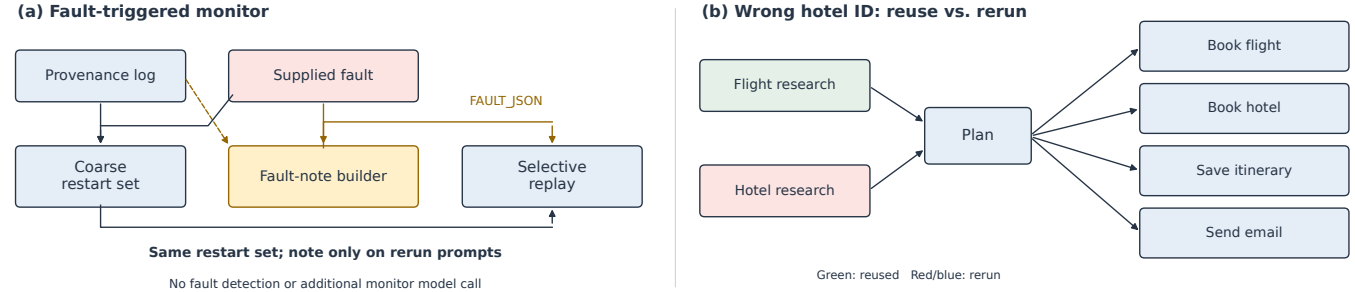


Figure 1. Current fault-note prototype **(a)** and replay steps for a wrong hotel ID **(b)**. Arrows show artifact dependencies, not execution order.

5. Initial Results

All methods recover identical faulty states; costs cover replay only. Appendix A lists coding injections; the repository experiment guide gives full run settings.

5.1 Travel: equal correctness, lower replay cost

Both clean tasks passed, and all 16 selected faulty runs failed without recovery. Every recovery policy repaired all 16 (Table 1). Since all methods reach the same task-success score, this small workload primarily distinguishes replay cost rather than recovery reliability. Relative to restart, coarse lineage used 45% fewer tokens and 43% less time while rerunning 34% fewer steps.

Method	Correct	Steps	Tokens	Sec.
Restart	16/16	7.0	6,678	17.4
Suffix	16/16	5.5	4,531	12.0
Coarse	16/16	4.6	3,650	9.9
Monitor	16/16	4.6	3,715	9.6
Support	16/16	4.5	3,578	9.7

Table 1. Qwen travel recovery. Costs are means over 16 cases; steps are out of seven.

5.2 Coding: lower cost, retained errors

On the coding tasks, selective replay reduced cost but retained errors that a restart could repair (Table 2). GPT-OSS coarse replay used 52% fewer tokens and 56% less time than restart, with nine fewer correct outcomes. The corresponding Muse outcomes were 87/95 for restart and 77/95 for coarse replay, with mean replay time reduced from 72.0 to 40.4 seconds. These are final-correctness counts, not rescue rates. On GPT-OSS, 33 injected runs failed before recovery; identifying which were rescued, and whether previously correct runs regressed, requires paired analysis. Both suffix and coarse failed all ten stale-report cases: they reran tests and review but retained the bad code. This exposes a limit of forward dependency traversal when repair requires reopening an upstream producer.

Model	Method	Correct	Steps	Tokens	Sec.
GPT-OSS	None	63/96	–	–	–
	Restart	95/96	9.0	3,340	31.8
	Suffix	86/96	5.4	1,845	16.9
	Coarse	86/96	5.0	1,607	13.9
Muse	None	57/95	–	–	–
	Restart	87/95	9.0	4,207	72.0
	Suffix	78/95	5.4	2,478	43.2
	Coarse	77/95	5.0	2,323	40.4

Table 2. MBPP outcomes after recovery, including cases already correct without recovery. Steps are out of nine. Muse coarse costs use 94 completed cost records; its correctness count includes one API error as a failure.

5.3 Monitor and support: mixed outcomes

We then examine whether the fault note and support links improve on coarse replay. On travel, the monitor changed mean time from 9.9 to 9.6 seconds and tokens from 3,650 to 3,715, without changing correctness or the restart set. A single trial per condition cannot establish a speedup. On synthetic graphs, monitor correctness fell from coarse's 17/17 to 14/17. The support rule saved one flight-booking step in the hotel-ID example, but failed cases that needed broader repair (Appendix C). Neither variant was evaluated on MBPP. The monitor therefore has no demonstrated coding benefit in this pilot. Its note can change regenerated outputs without improving them, so Phase 2 must evaluate correctness and cost together rather than treating reduced replay time as sufficient.

6. Limitations and Phase 2

These results establish baseline behavior, but leave three limits:

- **Coverage and uncertainty.** Ten MBPP tasks, two booking seeds, and one trial per condition do not establish stable gains. Accidental and adversarial faults are pooled.
- **Metrics and cost.** Replay correctness includes already-correct runs; paired rescue/regression rates remain to be computed. Always-verify uses a different denominator. Costs exclude the faulty forward run and do not isolate cleanup or tracking; hosted latency may include queueing.
- **Repair assumptions.** Fault locations are supplied. Context edges may overestimate dependencies; sparse support may omit needed repair. Simulated travel success does not erase irreversible harm, and the policy pilot observed no unauthorized attempts.

6.1 Recovery orchestrator

To move beyond fixed replay rules, we plan to mirror inter-agent handoffs into a central dependency record. After a fault is reported, the orchestrator will use that record to assign specific repair subtasks while preserving valid results. The aim is forward recovery from the current task state, rather than a full restart. We will measure recording overhead; asynchronous collection does not guarantee zero latency cost.

6.2 Fast and slow recovery

We will also explore how much reasoning to allocate to recovery. Inspired by the Talker–Reasoner separation of fast responses and deliberative planning [8], we will test a lightweight path for routine handoffs and a slower recovery planner for high-dependency or high-impact transfers. This is a proposed adaptation, not an implemented result. We will compare utility, harmful actions, and added latency/tokens against always-fast and always-slow policies. Slower reasoning may still be wrong; model training and changes to weights or KV state are outside scope.

6.3 Milestones

Table 3 summarizes completed work and the remaining project plan.

Phase	Milestone	Status
Phase 1: Baseline	Build workflows, fault injection, dependency tracking, and recovery baselines.	Completed
Phase 1: Exploration	Implement the fault-note prototype; collect initial correctness and cost measurements.	Completed
Week 1	Review orchestration, selective recovery, and fast/slow literature; refine the monitor design.	Planned
Week 2	Prototype recovery orchestration; compare selective repair with coarse replay and restart.	Planned
Week 3	Explore fast/slow routing and its correctness–cost tradeoff.	Exploratory
Final	Expand workloads, repeat trials, analyze limitations, and package reproducible results.	Planned
Extension	Investigate repair across branching and cyclic dependencies.	Optional

Table 3. Project milestones and current status.

The orchestrator and fast/slow routing are planned extensions; Phase 1 implements coarse replay and fault-note injection. Code, run instructions, and saved results are linked in the appendix.

References

[1] M. Cemri, M. Z. Pan, S. Yang, et al. Why Do Multi-Agent LLM Systems Fail? *NeurIPS Datasets and Benchmarks Track*, 2025.

[2] S. Zhang, M. Yin, J. Zhang, et al. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. *ICML*, 2025. arXiv:2505.00212.

[3] Y. Xie et al. Modeling and Mitigating Error Cascades in LLM-Based Multi-Agent Systems. arXiv:2603.04474, 2026.

[4] T. Sah et al. UndoBench: Separating Task Competence from Recovery Capability in Tool-Using AI Agents. arXiv:2610.05622, 2026.

[5] H. Garcia-Molina and K. Salem. Sagas. *ACM SIGMOD*, 1987.

[6] Y. Wang, C. Yu, J. Zhang, Y. Duan, M. Zheng, Z. Wu, K. Shi, and T. Cai. From Faulty Memories to Corrected Actions: Dependency-Guided Rollback Repair for Memory-Augmented Agents. arXiv:2608.10502, 2026.

[7] J. Austin, A. Odena, M. Nye, et al. Program Synthesis with Large Language Models. arXiv:2108.07732, 2021.

[8] K. Christakopoulou, S. Mourad, and M. Matarić. Agents Thinking Fast and Slow: A Talker-Reasoner Architecture. arXiv:2410.08328, 2024. https://arxiv.org/abs/2410.08328.

Appendix

Full setup, run commands, and configurations are in experiment guide; saved rows, traces, and source revisions are in saved artifacts. This appendix retains only evidence supporting the report's findings.

A. Coding Fault Coverage

Table 4 distinguishes scheduled attempts from applied injections on ten GPT-OSS tasks. Ineligible attempts are excluded from recovery scores.

Fault	Change	Category	Scheduled	Took effect
Bad patch	Replace the function body with `return None`, in pass 1 or 2.	Accidental	20	20
Flipped comparison	Change a comparison such as $>$ to $\geq$, in pass 1 or 2.	Accidental	20	10
Dropped requirement	Remove one requirement from the specification, in pass 1 or 2.	Accidental	20	6
Tester timeout	Replace test feedback with a simulated timeout, in pass 1 or 2.	Accidental	20	20
Instruction hijack	Instruct the coder to ignore the task and return `None`, in pass 1 or 2.	Adversarial	20	20
False test report	Report that faulty code passed its test, in pass 1.	Adversarial	10	10
Stale test report	Supply an earlier passing report for changed code, in pass 2.	Accidental	10	10
Total			**120**	**96**

Table 4. Coding injection coverage. The first five faults target both passes; false reports target pass 1 and stale reports pass 2.

B. Recovery Outcomes by Fault Placement

Each Table 5 cell gives correct outcomes after recovery and mean stages rerun out of nine, including runs already correct before recovery.

Fault and pass	Restart	Suffix	Coarse
Tester timeout, pass 1	10/10, 9.00	10/10, 7.00	10/10, 6.00
Tester timeout, pass 2	10/10, 9.00	10/10, 2.00	10/10, 2.00
Bad patch, pass 1	9/10, 9.00	10/10, 8.00	10/10, 7.00
Bad patch, pass 2	10/10, 9.00	10/10, 3.00	10/10, 3.00
Dropped requirement, pass 1	3/3, 9.00	3/3, 9.00	3/3, 9.00
Dropped requirement, pass 2	3/3, 9.00	3/3, 4.00	3/3, 4.00
Instruction hijack, pass 1	10/10, 9.00	10/10, 9.00	10/10, 9.00
Instruction hijack, pass 2	10/10, 9.00	10/10, 4.00	10/10, 4.00
False test report, pass 1	10/10, 9.00	10/10, 7.00	10/10, 6.00
Stale test report, pass 2	10/10, 9.00	0/10, 2.00	0/10, 2.00
Flipped comparison, pass 1	5/5, 9.00	5/5, 8.00	5/5, 7.00
Flipped comparison, pass 2	5/5, 9.00	5/5, 3.00	5/5, 3.00

Table 5. GPT-OSS recovery by placement. The stale-report cases show why retesting alone cannot repair retained bad code.

C. Supplementary Recovery Evidence

Table 6 records the small-case observations that motivate further work on monitor behavior and support sufficiency.

Case	Observation
Dropped requirement	On `clamp`, coarse versus monitor: 1,453 vs. 2,109 tokens and 8.2 vs. 13.9 s; both pass. Support reruns 2/9 stages and fails both `clamp` and `clip_text`.
Synthetic graphs	Correct outcomes: coarse 17/17, monitor 14/17, support 12/17.
Support plus note	Travel: 16/16 correct, 3,641 tokens and 9.0 s mean replay time.

Table 6. Supplementary pilot results; one observation per condition.